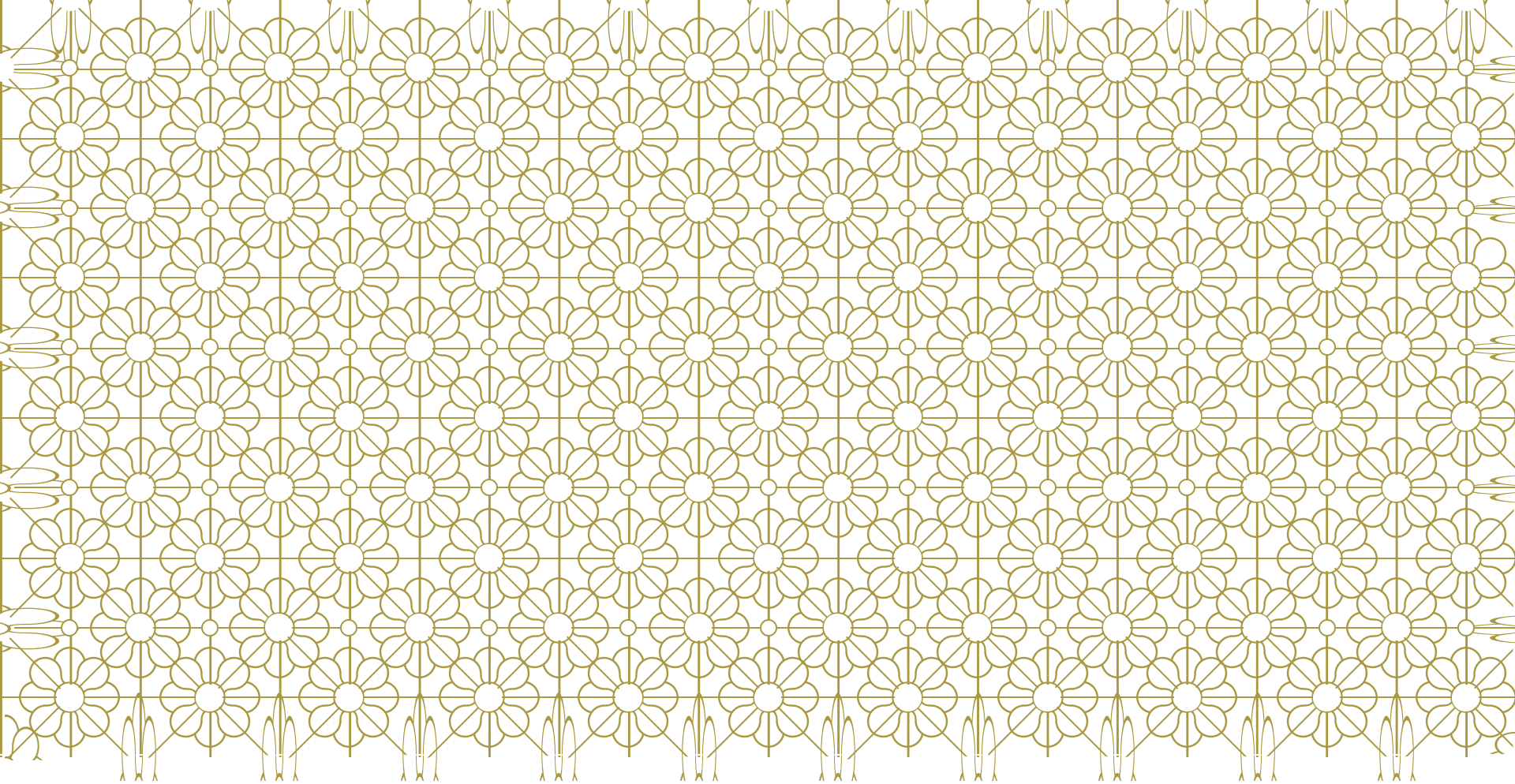
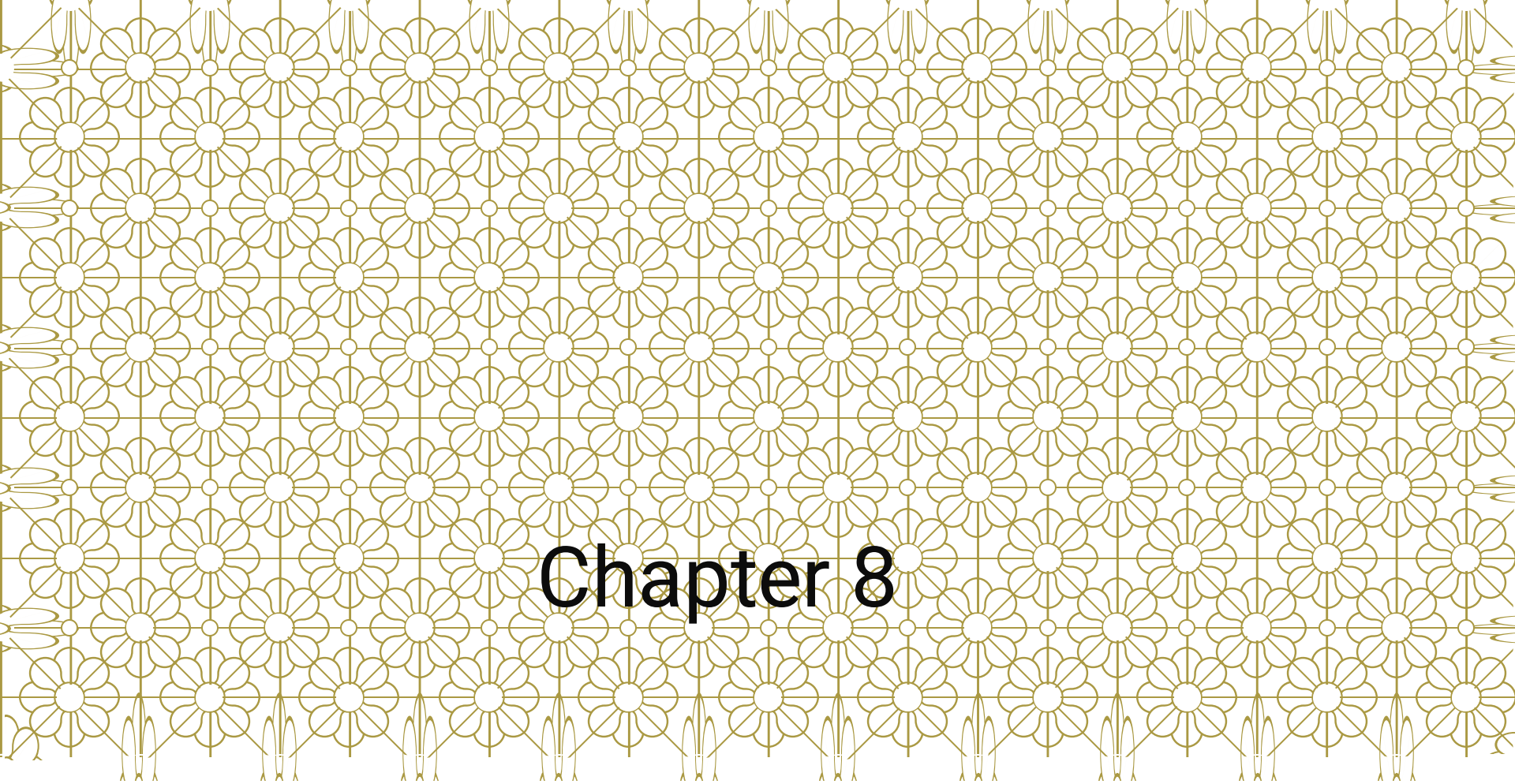




Algorithms analysis and Design

By : Dr. Basem
Alijla



Chapter 8

Dynamic Programming

OUTLINE

- Change-making problem
- Coin-collecting problem

The Knapsack Problem and Memory Functions

Warshall's and Floyd's Algorithms

Change-making problem

- Give change for amount n using the minimum number of coins of denominations $d_1 < d_2 < \dots < d_m$.
- Coin denominations used in the United State.
- Assuming availability of unlimited quantities of coins for each of the m denominations.
- $d_1 < d_2 < \dots < d_m$ where $d_1 = 1$.

Change-making problem

- Let $F(n)$ be the minimum number of coins whose values add up to n ;
- ❖ $F(0) = 0$.
- ❖ The amount n can only be obtained by adding one coin of denomination d_j to the amount $n - d_j$ for $j = 1, 2, \dots, m$ such that $n \geq d_j$.

$$F(n) = \min_{j : n \geq d_j} \{F(n - d_j)\} + 1 \text{ for } n > 0,$$

Change-making Algorithm

- ALGORITHM ChangeMaking($D[1..m]$, n)
- $F[0] = 0$
- for $i = 1$ to n do
 - $temp = \infty; j = 1$
 - while $j \leq m$ and $i \geq D[j]$ do
 - $temp = \min(F[i - D[j]], temp)$
 - $j = j + 1$
 - $F[i] = temp + 1$
- return $F[n]$

$O(nm)$

$$F(n) = \min_{j: n \geq d_j} \{F(n - d_j)\} + 1$$

Example 2: Change-making problem

- Application of Algorithm MinCoinChange to amount $n = 6$ and coin denominations 1, 3, and 4.

j	1	2	3
D	1	3	4

- $F[0] = 0$

n	0	1	2	3	4	5	6
F	0						

- $F[1] = \min\{F[1 - 1]\} + 1 = 1$

$$F(n) = \min_{j: n \geq d_j} \{F(n - d_j)\} + 1$$

Example 2: Change-making problem

- Application of Algorithm MinCoinChange to amount $n = 6$ and coin denominations 1, 3, and 4.

j	1	2	3
D	1	3	4

- $F[0] = 0$

n	0	1	2	3	4	5	6
F	0	1					

- $F[2] = \min\{F[2 - 1]\} + 1 = 2$

$$F(n) = \min_{j: n \geq d_j} \{F(n - d_j)\} + 1$$

Example 2: Change-making problem

- Application of Algorithm MinCoinChange to amount $n = 6$ and coin denominations 1, 3, and 4.

j	1	2	3
D	1	3	4

- $F[0] = 0$

n	0	1	2	3	4	5	6
F	0	1	2				

- $F[3] = \min\{F[3 - 1], F[3 - 3]\} + 1 = 1$

$$F(n) = \min_{j: n \geq d_j} \{F(n - d_j)\} + 1$$

Example 2: Change-making problem

- Application of Algorithm MinCoinChange to amount $n = 6$ and coin denominations 1, 3, and 4.

j	1	2	3
D	1	3	4

- $F[0] = 0$

n	0	1	2	3	4	5	6
F	0	1	2	1			

- $F[4] = \min\{F[4 - 1], F[4 - 3], F[4 - 4]\} + 1 = 1$

$$F(n) = \min_{j: n \geq d_j} \{F(n - d_j)\} + 1$$

Example 2: Change-making problem

- Application of Algorithm MinCoinChange to amount $n = 6$ and coin denominations 1, 3, and 4.

j	1	2	3
D	1	3	4

- $F[0] = 0$

n	0	1	2	3	4	5	6
F	0	1	2	1	1		

- $F[5] = \min\{F[5 - 1], F[5 - 3], F[5 - 4]\} + 1 = 2$

$$F(n) = \min_{j: n \geq d_j} \{F(n - d_j)\} + 1$$

Example 2: Change-making problem

- Application of Algorithm MinCoinChange to amount $n = 6$ and coin denominations 1, 3, and 4.

j	1	2	3
D	1	3	4

- $F[0] = 0$

n	0	1	2	3	4	5	6
F	0	1	2	1	1	2	2

- $F[6] = \min\{F[6 - 1], F[6 - 3], F[6 - 4]\} + 1 = 2$

Example 2: Change-making problem

- back trace the computations to see which of the denominations produced the minima

$$F(n) = \min_{j: n \geq d_j} \{F(n - d_j)\} + 1 \text{ for } n > 0,$$

j	1	2	3
D	1	3	4

n	0	1	2	3	4	5	6
F	0	1	2	1	1	2	2

$$F[6] = \min\{F[6 - 1], F[6 - 3], F[6 - 4]\} + 1 = F[3] + 1, d_2$$

$$F[3] = \min\{F[3 - 1], F[3 - 3]\} + 1 = F[0] + 1, d_2$$

OUTLINE

- Change-making problem
- Coin-collecting problem

The Knapsack Problem and Memory Functions

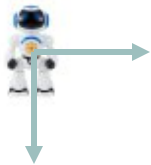
Warshall's and Floyd's Algorithms

Example 3: Coin-collecting problem

- Several coins are placed in cells of an $n \times m$ board, no more than one coin per cell.
- A robot, located in the upper left cell of the board, needs to collect as many of the coins as possible and bring them to the bottom right cell.
- the robot can move either one cell to the right or one cell down from its current location
- The robot picks up the coin at the visited cells

Example 3: Coin-collecting problem

- Design an algorithm to find the maximum number of coins the robot can collect and a path it needs to follow to do this.



Example 3: Coin-collecting problem

- Let $F(i,j)$ be the largest number of coins the robot can collect and bring to cell (i,j) in the i th row and j th column.
- The largest number of coins that can be brought to cell (i,j) :
 - from the adjacent cell $(i - 1, j)$ neighbor above it?
 - From the adjacent cell $(i, j - 1)$: the left neighbor ?

Example 3: Coin-collecting problem

- The recurrence:

$$F(i, j) = \max\{F(i-1, j), F(i, j-1)\} + c_{ij}$$

$$\text{for } 1 \leq i \leq n, 1 \leq j \leq m$$

where $c_{ij} = 1$ if there is a coin in cell (i,j) , and $c_{ij} = 0$ otherwise

$F(0, j) = 0$ for $1 \leq j \leq m$ and $F(i, 0) = 0$ for $1 \leq i \leq n$.

Change-making Algorithm

- ALGORITHM RobotCoinCollection($C[1..n, 1..m]$)
- $F[1,1] \leftarrow C[1,1]$
- for $j \leftarrow 2$ to m do
 - $F[1, j] \leftarrow F[1, j - 1] + C[1, j]$ // first row
- for $i \leftarrow 2$ to n do
 - $F[i, 1] \leftarrow F[i-1, 1] + C[i, 1]$ // first column $O(nm)$
 - for $j \leftarrow 2$ to m do
 - $F[i, j] \leftarrow \max(F[i - 1, j], F[i, j - 1]) + C[i, j]$
- return $F[n, m]$

Example 3: Coin-collecting problem

- C

F		1	2	3	4	5	6
1	0						
2							

Example 3: Coin-collecting problem

- C

C₁₂

F		1	2	3	4	5	6
1	0	← +C ₁₂					
2							

Example 3: Coin-collecting problem

- C

C₂₁

		F					
		1	2	3	4	5	6
1		0	0	0	0	1	1
		↑					
		+C ₂₁					
2							

Example 3: Coin-collecting problem

- C

F		1	2	3	4	5	6
1	0	0	0	0	1	1	
	0						
	0						
	0						
2	1						

Example 3: Coin-collecting problem

- C

C_{22}

		F					
		1	2	3	4	5	6
1		0	0	0	0	1	1
		0	max + C_{22}				
		0					
		0					
		1					
2							

Example 3: Coin-collecting problem

- C

C_{23}

F		1	2	3	4	5	6
1		0	0	0	0	1	1
		0	1	max + C_{23}			
		0					
		0					
2		1					

Example 3: Coin-collecting problem

- C

C_{24}

F		1	2	3	4	5	6
1		0	0	0	0	1	1
		0	1	1	max + C_{24}		
		0					
2		0					
		1					

Example 3: Coin-collecting problem

- C

C₂₅

F		1	2	3	4	5	6
1		0	0	0	0	1	1
		0	1	1	2	max +C ₂₅	
		0					
		0					
	2	1					

Example 3: Coin-collecting problem

- C

C_{26}

F		1	2	3	4	5	6
1		0	0	0	0	1	1
		0	1	1	2	2	max + C_{26}
2		0					
		0					
		1					

Example 3: Coin-collecting problem

- C

F		1	2	3	4	5	6
1		0	0	0	0	1	1
		0	1	1	2	2	2
2		0					
		0					
		1					

Example 3: Coin-collecting problem

- C

F		1	2	3	4	5	6
1		0	0	0	0	1	1
		0	1	1	2	2	2
		0	1	1	3	3	4
2		0					
		1					

Example 3: Coin-collecting problem

- C

F		1	2	3	4	5	6
1		0	0	0	0	1	1
		0	1	1	2	2	2
		0	1	1	3	3	4
		0	1	2	3	3	5
2		1					

Example 3: Coin-collecting problem

- C

F		1	2	3	4	5	6
1		0	0	0	0	1	1
		0	1	1	2	2	2
		0	1	1	3	3	4
		0	1	2	3	3	5
2		1	1	2	3	4	5

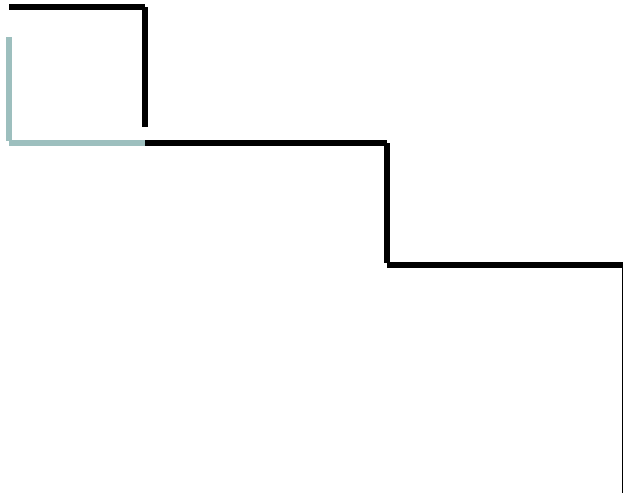
Example 3: Coin-collecting problem

- Tracing the computations backward makes it possible to get an optimal path:
- if $F(i - 1, j) > F(i, j - 1)$, an optimal path to cell (i, j) must come down **from the adjacent cell above it**
- if $F(i - 1, j) < F(i, j - 1)$, an optimal path to cell (i, j) must come from **the adjacent cell on the left**.
- if $F(i - 1, j) = F(i, j - 1)$, it can reach cell (i, j) **from either direction**.

Example 3: Coin-collecting problem

- Tracing the computations backward makes it possible to get an optimal path:

C



F	1	2	3	4	5	6
1	0	0	0	0	1	1
	0	1	1	2	2	2
	0	1	1	3	3	4
	0	1	2	3	3	5
2	1	1	2	3	4	5

OUTLINE

- Change-making problem
- Coin-collecting problem

The Knapsack Problem and Memory Functions

Warshall's and Floyd's Algorithms